DISK commands Linux:

🔹 1. `df` Command (Disk Free)

What

`df` shows **filesystem-level disk usage** — how much space is **used, available, and mounted**.

When

Use it when:

- Server is showing **"No space left on device"**
- Application logs are failing
- Kubernetes pods crashing due to disk issues

Where

- Used in **production servers**, EC2 instances, containers
- Helps in checking mounted volumes like `/`, `/var`, `/home`

How

```
df -h
```

Example Output

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   45G   3G   94% /
```

Why (Important Insight)

- Shows **filesystem full**, not individual files
- If `Use%` is **>90%**, it's risky

👉 Real-world scenario:
Your Java app stops writing logs → you run `df -h` → root `/` is 100% full → production issue identified.

🔹 2. du Command (Disk Usage)

What

`du` shows **file/folder-level disk usage** — where exactly space is being consumed.

When

Use it when:

- `df` shows disk is full but you don't know **which folder**
- Debugging **log explosion**, temp files, dumps

Where

- Inside directories like:
  - `/var/log`
  - `/tmp`
  - application folders

How

```
du -sh *
```

Useful Variations

```
du -sh /var/log        # total size
du -h --max-depth=1    # folder-wise breakdown
```

The command `du -sh /var/log` is used to check the total disk usage of the `/var/log` directory. Here, `du` means disk usage, `-s` means "summary" (so it shows only the final total instead of listing every file), and `-h` makes the output human-readable (like MB, GB instead of bytes). In simple terms, it tells you how much space all log files together are consuming. For example, if it shows `500M /var/log`, it means all logs inside that folder are using 500 MB.

The command `du -h --max-depth=1` is used to see disk usage of directories one level deep from the current location. Here, `--max-depth=1` means it will show sizes of each folder inside the current directory, but not go deeper into subfolders. This is useful when you want to quickly identify which top-level folder is taking more space. For example, if you run it in `/var`, it may

show `/var/log = 500M`, `/var/cache = 1.2G`, helping you understand where space is being used.

In real-world scenarios (especially in SRE or production), these commands are used to quickly debug disk space issues. For instance, if a server disk is full, you can run `du -h --max-depth=1 /var` to find that `/var/log` is huge, and then `du -sh /var/log` to confirm total usage before cleaning old logs.

Example Output

```
2.5G logs/
500M temp/
1.2G cache/
```

Why (Important Insight)

- Helps identify **which folder is eating space**
- Works *inside* filesystem (unlike df)

👉 Real-world scenario:
`df -h` shows full disk → `du -sh /var/log/*` → huge log file found → you clean it → issue resolved.

🔹 3. Inode Concept

What

An **inode** is a metadata structure storing:

- File size
- Permissions
- Owner
- Location on disk

⚠️ Important:
Disk can be full even if space is available — due to inode exhaustion

When

Use when:

- Error: `No space left on device` but `df` shows free space
- Happens with **millions of small files**

Where

- Log-heavy systems
- Cache-heavy apps
- Microservices writing many small temp files

How

```
df -i
```

Example Output

```
Filesystem     Inodes  IUsed   IFree IUse% Mounted on
/dev/sda1      1000000 1000000     0  100% /
```

Why (Critical Insight)

- Each file = 1 inode
- If inodes are exhausted → cannot create new files

Column	Meaning
Filesystem	Disk/partition name
Inodes	Total inodes available
IUsed	How many inodes are used
IFree	How many are free
IUse%	Percentage of inode usage
Mounted on	Mount point

👉 Real-world scenario:
Your app crashes → `df -h` shows space available
But `df -i` shows 100% → millions of small temp files → root cause found

🔥 Interview-Level Understanding (Very Important)

Command	Focus	Level
`df`	Filesystem usage	High-level
`du`	Folder/file usage	Detailed
`inode`	File count limit	Deep system

🔥 Production Debug Flow

When disk issue happens:

Step 1:

```
df -h
```

➡️ Check if disk is full

Step 2:

```
df -i
```

➡️ Check inode exhaustion

Step 3:

```
du -sh /*
```

➡️ Find heavy directories

🔥 Real Production Example

- Application Logs not writing
- Error: `No space left on device`

You check:

1. `df -h` → 95% full
2. `du -sh /var/log` → 20GB logs
3. Clean logs → issue fixed

DEEPER UNDERSTANDING:

🔹 Step 1: `df` Command (Disk Free)

What

`df` shows **disk space at filesystem level**.
It tells you:

- Total size
- Used space
- Free space
- Mount location

Think of it like:
👉 "Which disk is full?

How (Basic Command)

```
df -h
```

Why `-h`?

- Makes output **human-readable** (GB, MB instead of bytes)

Example Output

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   45G   3G   94% /
tmpfs           2.0G  200M  1.8G  10% /run
```

Understanding Output (Important)

- **Filesystem** → Disk/partition name
- **Size** → Total disk size
- **Used** → Space used
- **Avail** → Free space
- **Use%** → Percentage used
- **Mounted on** → Where it is attached

When to Use

You'll use `df` when:

Application throws:
No space left on device

- Logs stop writing
- Server becomes slow unexpectedly

Why It Matters

👉 It tells you **problem exists or not**

But:
❌ It does NOT tell *who is causing it*
(That's where `du` comes next)

🔥 Real Production Scenario

Your Java app logs error:

```
java.io.IOException: No space left on device
```

You run:

```
df -h
```

You see:

```
/dev/sda1   100%   /
```

👉 Boom — root disk is full → **problem identified**

⚠️ Important Edge Case

Sometimes:

- `df` shows full
- But you deleted files

Still no space freed?

👉 This happens due to **open file handles (we'll cover in `lsof` later)**

🧠 Mini Task (Do This Now)

Run this on your system:

```
df -h
```

Try to answer:

- Which partition is most full?
- What is your root (`/`) usage?

Once you're done, tell me:

🔹 Step 2: du Command (Disk Usage)

What

`du` shows **how much space each file/folder is using inside a directory**.

Think of it like:
👉 `df` told you *"disk is full"*
👉 `du` tells you *"who filled it"*

How (Basic Command)

```
du -sh *
```

Meaning:

- `-s` → summary (total per folder)
- `-h` → human-readable
- **Example Output**

```
2.5G logs/
500M temp/
1.2G cache/
```

👉 Now you clearly see:

- `logs/` is the main problem (2.5GB)

🔥 Important Variations (Very Useful in Production)

1. Check a specific directory

```
du -sh /var/log
```

2. Breakdown inside a directory

```
du -h --max-depth=1 /var
```

👉 Output:

```
3G /var/log
1G /var/lib
200M /var/tmp
```

3. Find largest files/folders

```
du -ah / | sort -rh | head -10
```

👉 This shows **top 10 biggest space consumers**

The command `du -ah / | sort -rh | head -10` is used to find the top 10 largest files and directories in the entire system. Here, `du -ah /` lists disk usage of all files and folders starting from root (`/`), where `-a` includes individual files and `-h` makes sizes human-readable. This gives a very detailed list of everything consuming space.

The output of `du` is then passed to `sort -rh`, which sorts the data in reverse order (largest first) while understanding human-readable sizes like GB, MB. This ensures that the biggest files or directories appear at the top of the list.

Finally, `head -10` limits the output to only the top 10 entries, so you don't get overwhelmed with too much data. In a real-world scenario, this command is extremely useful when a server disk is full and you need to quickly identify which files or directories are consuming the most space, for example finding a huge log file or a backup folder accidentally growing to several GBs.

When to Use (Real Scenario)

Use `du` when:

- `df -h` shows disk is full
- You need to find:
  - Large logs
  - Temp files
  - Cache buildup
  - Dump files

Why It Matters

👉 Without `du`, you only know *there is a problem*
👉 With `du`, you know *exact root cause*

🔥 Real Production Scenario

Step 1:

```
df -h
```

→ Disk is 95% full

Step 2:

```
du -sh /var/log/*
```

Output:

```
10G app.log
500M system.log
```

👉 Root cause: `app.log` is too big

Fix:

```
truncate -s 0 app.log
```

👉 Disk issue resolved

⚠️ Common Mistakes

❌ Running du on root /

```
du -sh /
```

👉 This is slow and heavy

✔ Instead:

```
du -h --max-depth=1 /
```

🧠 Key Difference (Must Remember)

Command	Purpose
`df`	Shows **disk full or not**
`du`	Shows **who is using space**

🧠 Mini Task (Do This Now)

Run:

```
du -h --max-depth=1 /
```

Try to answer:

- Which directory is largest?
- Is `/var` or `/home` consuming more?

🔹 Step 3: Inodes (Hidden Disk Problem)

What

An **inode** is a data structure that stores **metadata about a file**, such as:

- File size
- Permissions
- Owner
- Disk location

👉 Important:
Every file = 1 inode

🔥 Simple Understanding

Think of disk like:

- **Space (GB)** → how big files are
- **Inodes** → how many files you can create

👉 You can run out of **file count** even if space is available

How to Check Inodes

```
df -i
```

Example Output

```
Filesystem     Inodes  IUsed   IFree IUse% Mounted on
/dev/sda1      1000000 1000000     0  100% /
```

Understanding Output

- **Inodes** → Total number of file slots
- **IUsed** → Used inodes
- **IFree** → Remaining
- **IUse%** → Usage percentage

👉 If `IUse% = 100%` → 🚨 **You cannot create new files**

When to Use (Critical Scenario)

Use `df -i` when:

You see:
No space left on device

BUT `df -h` shows **free space available**

🔥 Real Production Scenario

Situation:

- App crashes

Error:
No space left on device

You check:

```
df -h
```

👉 Shows:

Avail: 10GB free

❓ Confusing right?

Then you check:

```
df -i
```

👉 Output:

IUse% = 100%

👉 Root cause:
Too many small files (inode exhausted)

🔥 Where This Happens in Real World

- Log-heavy apps (millions of tiny logs)
- Cache directories
- Temp folders (`/tmp`)
- Microservices generating small files

How to Investigate Further

Find directories with too many files:

```
find / -xdev -type f | wc -l
```

Or per folder:

```
find /var/log -type f | wc -l
```

How to Fix

👉 Remove unnecessary small files:

```
rm -rf /tmp/*
```

👉 Clean logs:

```
rm -rf /var/log/old_logs/*
```

⚠️ Important Insight

Problem Type	Command
Disk space full	`df -h`
File count full (inode)	`df -i`

🔥 SRE Debug Flow (Now Complete)

When you see disk issue:

Step 1:

```
df -h
```

Step 2:

```
df -i
```

Step 3:

```
du -h --max-depth=1 /
```

🧠 Mini Task (Do This)

Run:

```
df -i
```

Check:

- What is your inode usage?
- Is anything close to 100%?

🚀 Next (Advanced Disk Debugging – Very Important)

Now we go deeper into **real SRE-level debugging tools**:

👉 Next step:

- `lsof` (🔥 deleted files still consuming space)
- `lsblk` (disk structure)
- `mount` / `umount`
- `iostat` (disk performance)

🔹 Step 4: `lsof` (List Open Files) — 🔥 Critical

What

`lsof` shows **which processes are using which files**.

👉 Full form:
lsof = list open files

🔥 Why This Is Important (Core Problem)

Sometimes:

- You delete a file
- But **disk space is NOT freed**

❓ Why?

👉 Because a process is still **holding the file open**

How to Use

```
lsof
```

But this gives huge output.

🔥 Most Important Command (SRE Use Case)

```
lsof | grep deleted
```

What This Shows

- Files that are:
  - ❌ Deleted from disk
  - ✅ Still used by a running process

👉 These files still **occupy disk space**

🔥 Real Production Scenario (Very Important)

Step 1:

Disk is full:

```
df -h
```

Step 2:

You delete logs:

```
rm app.log
```

Step 3:

Check again:

```
df -h
```

👉 😳 Still full!

Step 4:

Now run:

```
lsof | grep deleted
```

👉 Output:

```
java  1234  user  5u  REG  /var/log/app.log (deleted)
```

🔥 Root Cause

👉 Java process is still holding the file in memory

So:

- File is deleted from directory
- But still exists in **process file descriptor**

How to Fix

Option 1: Restart process (most common)

```
kill -9 1234
```

👉 This releases the file → disk space freed

Option 2: Restart service

```
systemctl restart your-app
```

🔥 When to Use `lsof`

Use it when:

- Disk is full
- You already deleted files
- But space is NOT freeing

⚠️ Important Insight

Situation	Problem
File deleted	Directory entry gone
Process running	File still in memory
Result	Disk space NOT freed

🔥 Extra Useful Commands

Check specific process

```
lsof -p 1234
```

Check specific file

```
lsof /var/log/app.log
```

🧠 Simple Analogy

Think like this:

- You deleted a file 📄
- But someone is still reading it 👀

👉 Until they stop → file still “exists” internally

🧠 Mini Task (Important)

Run:

```
lsof | grep deleted
```

If nothing appears → good (no issue)

🔥 SRE-Level Debug Flow (Now Strong)

1. `df -h` → disk full?
2. `du` → who is using space?
3. `df -i` → inode issue?
4. `lsof` → deleted file still used?

👉 This is **real production debugging mindset**

🔹 Step 5: `lsblk` (List Block Devices)

What

`lsblk` shows **all disks, partitions, and how they are connected/mounted**.

👉 Think of it like:

- `df` → shows usage
- `lsblk` → shows **structure**

How to Use

```
lsblk
```

Example Output

```
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sda      8:0    0   50G  0 disk
├─sda1   8:1    0   48G  0 part /
└─sda2   8:2    0    2G  0 part [SWAP]
```

Understanding Output

- **sda** → Main disk
- **sda1, sda2** → Partitions
- **TYPE**
  - `disk` → physical disk
  - `part` → partition
- **MOUNTPOINT**
  - `/` → root filesystem
  - `[SWAP]` → swap memory

🔥 Why It Matters

👉 Helps you understand:

- How many disks are attached
- Which partition is mounted where
- Whether new disk is detected or not

🔥 Real Production Scenario

You attach a new disk in AWS (EBS volume)

You run:

```
lsblk
```

👉 You see:

```
sdb   100G   disk
```

👉 But no mount point

👉 Means:

- Disk is attached
- But NOT mounted yet

🔹 Step 6: `mount` (Attach Filesystem)

What

`mount` is used to **attach a disk/partition to a directory**

👉 In Linux:

- Disk is useless until mounted

How

```
mount /dev/sdb1 /data
```

👉 Meaning:

- Attach partition `sdb1`
- Make it accessible at `/data`

🔥 Example

After mounting:

```
lsblk
```

Now shows:

```
sdb1   100G   part   /data
```

🔹 Step 7: `umount` (Detach Filesystem)

What

`umount` removes the disk from system access

How

```
umount /data
```

👉 Now `/data` is no longer connected to disk

⚠️ Important Rules

❌ You cannot unmount if:

- Files are in use
- Process is accessing it

👉 You may need:

```
lsof /data
```

to check usage

🔥 Real Production Scenario

Situation:

- New disk added for logs

Steps:

1. `lsblk` → verify disk
2. `mount` → attach it
3. Move logs to new disk

👉 Prevents root disk from filling

🧠 Simple Analogy

- Disk = Hard drive 📦
- Mount = Plugging it into system 🔌
- Mount point = Folder where it appears 📁

🔥 Key Difference

Command	Purpose
`lsblk`	Show disk structure
`mount`	Attach disk
`umount`	Detach disk

🧠 Mini Task

Run:

```
lsblk
```

Try to answer:

- How many disks do you see?
- Which one is mounted at `/`?

Got it—let's revisit **`iostat`** properly and make it crystal clear with real-world understanding.

🔹 `iostat` (Disk I/O Statistics)

What

`iostat` is used to **monitor disk performance**.

👉 It tells you:

- How busy your disk is
- How fast data is being read/written
- Whether disk is causing **application slowness**

Why It Matters (Very Important)

In production, many times:

- CPU looks fine ✅
- Memory looks fine ✅
- But app is still slow ❌

👉 Root cause = **Disk I/O bottleneck**

How to Run

```
iostat -x 1
```

Meaning:

- `-x` → extended stats (important for debugging)
- `1` → refresh every 1 second (live monitoring)

Sample Output

```
Device:    tps    rkB/s    wkB/s    await    %util
sda        120    2000     5000     15       95
```

🔥 Important Fields (Focus Here)

1. %util (MOST IMPORTANT)

👉 Shows how busy the disk is

Value	Meaning

< 50%	Healthy
50–80%	Moderate load
> 90%	🚨 Disk is overloaded

👉 If `%util = 100%`
➡️ Disk is fully busy → requests are waiting → system slows down

2. await (Response Time)

👉 Time taken for each disk request

Value	Meaning
< 10 ms	Good
10–50 ms	Okay
> 100 ms	🚨 Slow disk

👉 High `await` = disk is slow

3. tps (Transactions Per Second)

👉 Number of read/write operations per second

- High TPS = heavy load
- But not always bad (depends on disk capacity)

4. rkB/s & wkB/s

👉 Data read/write speed

- rkB/s → read speed
- wkB/s → write speed

🔥 Real Production Scenario

Scenario 1: App is Slow

- API latency increased
- CPU = 20%
- Memory = OK

👉 You run:

```
iostat -x 1
```

Output:

```
%util = 99%
await = 200ms
```

👉 Root cause:
Disk is overloaded and slow

Scenario 2: Database Slow

- Queries taking long time

👉 Check:

```
iostat -x 1
```

If:

- `%util` high
- `await` high

👉 DB is waiting for disk → performance issue

🔥 How to Interpret Quickly (Interview Trick)

When you see `iostat`, just focus on:

👉 **2 things only**

1. `%util`
2. `await`

Quick Decision Rule

- `%util high` → disk overloaded
- `await high` → disk slow

👉 Both high = 🚨 serious issue

🔥 How to Fix (Real SRE Actions)

1. Reduce Disk Load

- Reduce logging
- Optimize queries
- Remove unnecessary writes

2. Move Data

- Logs → separate disk
- DB → dedicated storage

3. Upgrade Disk

- HDD → SSD
- Normal → high IOPS (cloud)

4. Use Caching

- Reduce frequent disk reads

🧠 Simple Analogy

- Disk = worker 👷
- Requests = tasks

👉 If worker is busy 100% → new tasks wait → slow system

🧠 Mini Practice

Run:

```
iostat -x 1
```

Observe:

- Which device (`sda`, `sdb`) is busiest?
- What is `%util`?
- What is `await`?

🔥 Interview Answer (Perfect Way)

If interviewer asks:

👉 “How do you detect disk bottleneck?”

You say:

> “I use `iostat -x 1` to monitor disk performance. I mainly check `%util` to see if disk is saturated and `await` to understand latency. If both are high, it indicates a disk bottleneck affecting application performance.”

🔹 🟢 Beginner Level (0–2 Years)

Q1. Disk Full Issue

👉 Your application suddenly stops writing logs. Error:

No space left on device

What will you do?

Expected flow:

- Run:

```
df -h
```

- If full → identify usage:

```
du -h --max-depth=1 /
```

- Clean logs / temp files

Q2. Disk Full but Space Available

👉 `df -h` shows free space, but still same error.

What now?

Expected:

- Check inode:

```
df -i
```

👉 Root cause: inode exhaustion

Q3. Logs Deleted but Space Not Freed

👉 You deleted large log files, but disk is still full.

What will you check?

Expected:

```
lsof | grep deleted
```

👉 Restart process holding file

Q4. Find Largest Files

👉 Disk is 90% full. You need to quickly find biggest files.

Expected:

```
du -ah / | sort -rh | head -10
```

🔹 🟡 Intermediate Level (2–4 Years)

Q5. New Disk Not Visible

👉 You attached a new disk in cloud, but app cannot use it.

What will you do?

Expected:

```
lsblk
```

👉 If not mounted:

```
mount /dev/sdb1 /data
```

Q6. Cannot Unmount Disk

👉 You try:

```
umount /data
```

But it fails.

Why?

Expected:

- Some process is using it

Check:

```
lsof /data
```

Q7. Application Slow but CPU Normal

👉 CPU = 20%, Memory OK, but app is slow.

What will you check?

Expected:

```
iostat -x 1
```

👉 If `%util` ~100% → disk bottleneck

Q8. Too Many Small Files

👉 System crashes due to temp files.

How to identify?

Expected:

```
find /tmp -type f | wc -l
```

🔹 🟠 Advanced Level (4–7 Years)

Q9. Production Incident (Full Debug Flow)

👉 Alert:
“API latency increased from 100ms → 5s”

How do you debug?

Expected thinking:

1. Check CPU:

```
top
```

2. Check memory:

```
free -m
```

3. Check disk:

```
iostat -x 1
```

👉 If `%util` high → disk bottleneck

Q10. Hidden Disk Usage Issue

👉 `du` shows low usage, but `df` shows full disk.

Why mismatch?

Expected:
👉 Deleted files still open

Check:

```
lsof | grep deleted
```

Q11. Kubernetes Disk Issue

👉 Pods are restarting with:

```
Evicted: DiskPressure
```

What will you check?

Expected:

- Node disk:

```
df -h
```

- Container logs:

```
du -sh /var/lib/docker
```

👉 Clean unused images:

```
docker system prune
```

Q12. Database Slow Suddenly

👉 DB queries slow, CPU low.

Root cause?

Expected:

- Disk I/O bottleneck

Check:

```
iostat -x 1
```

👉 High `await` → slow disk

Q13. Log Explosion Issue

👉 One service is generating huge logs rapidly.

What will you do?

Expected:

- Identify:

```
du -sh /var/log/*
```

- Fix:
    - Rotate logs
    - Limit logging level

Q14. Real Trick Question

👉 You removed files, restarted service, but still disk is full.

What could be wrong?

Expected answers:

- Multiple processes holding files
- Wrong partition cleaned
- Mounted disk confusion (`lsblk`)

Q15. Design-Level Question (7 Years)

👉 How will you **prevent disk issues in production?**

Expected answer:

- Log rotation (`logrotate`)
- Monitoring:
  - Disk usage alerts (80%, 90%)
- Separate disks:
  - `/` → system
  - `/var/log` → logs
- Auto-clean scripts
- Use cloud auto-scaling / bigger volumes

🔥 Interview Pattern (Important Insight)

Interviewers are testing:

Skill	What they check

Debugging	Step-by-step thinking
Tools	`df`, `du`, `lsof`, `iostat`
Root cause	Not just commands
Experience	Real incidents

🔥 How You Should Answer (Golden Rule)

Always answer like:

👉 "First I will check…"
👉 "Then based on output…"
👉 "If this happens, I will…"

Example:

> "First I'll run `df -h` to check disk usage. If disk is full, I'll use `du` to identify large directories. If space is still not freed, I'll check `lsof` for deleted files…"

👉 This structured thinking = **selected candidate**